Optimización y auditoría de un modelo de aprobación de crédito en German Credit

German Credit 6.2

Atlantis University

MAI 540: Machine Learning

Kevin A. García Gallardo

13 de octubre de 2026

Optimización y auditoría de un modelo de aprobación de crédito en German Credit

Se optimizó y auditó el modelo de aprobación de crédito de la Tarea 6.1 con German Credit (1,000 solicitudes, 20 variables, 30 % de créditos malos; Hofmann, 1994). Aprobar un crédito malo suele costar más que rechazar uno bueno, así que, junto con la exactitud, se reporta la proporción de créditos malos detectados (recall de la clase mala), la métrica prioritaria; no hay matriz de costos y no se supuso ninguna. Edad y estado civil se usaron solo para auditar, no para decidir (18 variables).

Método

Todos los modelos usaron la misma validación cruzada estratificada de 5 pliegues (semilla 42) y un *Pipeline* con la codificación ajustada dentro de cada pliegue (Pedregosa et al., 2011); el protocolo reproduce las exactitudes de la Tarea 6.1 (.698 y .759). El árbol se afinó con *GridSearchCV* sobre *max_depth* ∈ {1, 2, 3, 4, 5, 7, 10, sin límite} y *min_samples_leaf* ∈ {1, 2, 5, 10, 20} (40 combinaciones), eligiendo por exactitud. Como el mejor puntaje de una búsqueda es optimista (Cawley y Talbot, 2010), el número honesto sale de una validación cruzada anidada: la búsqueda se repite dentro de cada pliegue externo con solo su entrenamiento. Random Forest e *HistGradientBoostingClassifier* se evaluaron con parámetros por defecto.

Búsqueda y medición honesta

La ganadora fue max_depth = 5 y min_samples_leaf = 5: exactitud .731 en la búsqueda frente a .690 del árbol por defecto (+4.1 pp). Con validación anidada, el árbol afinado obtuvo .710 (DE = .033); la búsqueda sobrestimó 2.1 pp y la mejora honesta es +2.0 pp. La ganadora cambió entre pliegues (profundidad 3 a 7, hoja mínima 10 a 20): el óptimo es inestable. Además, afinar por exactitud redujo los malos detectados de .480 a .357.

En la Tabla 1, HistGradientBoosting y Random Forest superan la línea base en 6.6 y 6.5 pp; el árbol afinado, en 1.0 pp. Su exactitud es equivalente (.766 y .765), pero HistGradientBoosting detecta .510 de los malos frente a .417.

Tabla 1

Comparación de modelos con el mismo protocolo (validación cruzada de 5 pliegues)

Modelo	Exactitud M (DE)	Malos detectados M (DE)	Ajuste (s/pliegue)
Árbol por defecto	.690 (.013)	.480 (.038)	0.02
Árbol afinado (anidada)	.710 (.033)	.357 (.115)	4.86*
Árbol afinado (búsqueda, optimista)	.731	.323	—
Random Forest	.765 (.020)	.417 (.012)	0.18
HistGradientBoosting	.766 (.011)	.510 (.035)	0.26

Nota. M = media; DE = desviación estándar muestral entre pliegues. Línea base «siempre bueno» = .700. * Incluye la búsqueda completa dentro de cada pliegue; ajustar una vez la configuración ganadora toma 0.012 s.

Curva de aprendizaje

La Tarea 6.1 no incluyó curva de aprendizaje; el diagnóstico se reconstruyó con sus modelos por defecto y 20 variables, con el mismo código y la misma validación cruzada (Figura 1). La brecha entre entrenamiento y validación fue .302 en el árbol y .246 en Random Forest; en HistGradientBoosting es .234 (−1.2 pp frente a Random Forest; con 20 variables, .242); no se redujo de forma relevante porque aprende casi por completo el entrenamiento (1.000). Solo el árbol afinado la reduce (.051), a costa de rendimiento. La validación sigue subiendo (+0.95 pp por cada 100 casos entre 480 y 800) sin llegar a una meseta: más datos probablemente ayudarían, poco y con incertidumbre (DE de 1 a 3 pp entre pliegues).

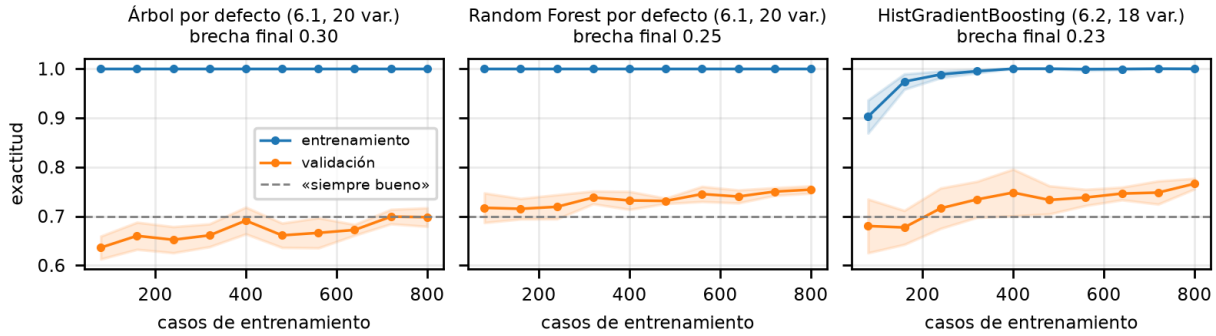


Figura 1. *Curvas de aprendizaje: diagnóstico de la Tarea 6.1 (reconstruido) y modelo elegido.* Banda = ±1 DE entre pliegues; línea punteada = línea base «siempre bueno».

Auditoría y pruebas

Tabla 2

Malos detectados por subgrupo, HistGradientBoosting (predicciones fuera de pliegue)

Variable	Rango de recall	Mayor desviación	Dif. vs global	Umbral .10
Edad (4 grupos)	.473–.574	<25 (61 malos)	+0.064	dentro
Estado civil y sexo (4)	.300–.541	male div/sep (20 malos)	−0.210	supera
Vivienda (3)	.462–.614	for free (44 malos)	+0.104	supera

Nota. Recall global = .510. Dif. = recall del subgrupo − global; mínimo 20 malos.

La Skill de auditoría del Módulo 4 (v2.0) dio: métricas PASA; partición FALLA; fuga FALLA; subgrupos FALLA; datos de entrada NO SE PUEDE DETERMINAR; variables no admisibles PASA. Superan el umbral (Tabla 2) «male div/sep» en estado civil (.300; 6 de 20 malos: muestra pequeña) y «for free» en vivienda (.614; excede por .0036). Partición y fuga fallan por diseño: el diagnóstico de la 6.1 usa otras variables y el modelo se eligió entre tres con la misma validación cruzada, sin conjunto reservado. Las 9 pruebas de pytest pasan; dos exigen superar la línea base de .700 y una DE entre pliegues ≤ .05, umbral de 1.5 veces el error estándar de un pliegue de 200 casos (√(.70 × .30 / 200) = .032).

Recomendación

Llevaría a producción HistGradientBoosting, como apoyo a la decisión y no como decisión automática. Rendimiento: exactitud .766 (DE = .011, la menor) y .510 de malos detectados, la más alta; Random Forest empata en exactitud pero detecta .417, y el árbol afinado es inestable (DE del recall = .115). Costo: 0.26 s por ajuste frente a 4.9 s del árbol afinado con su búsqueda. Interpretabilidad: menor que la de un árbol; la importancia por permutación indica que usa sobre todo checking_status, duration y credit_amount (caídas de exactitud de .062, .043 y .031), ninguna sensible. El error que más me preocupa es aprobar un crédito malo: el modelo deja pasar 147 de 300 (49 %). Lo reflejé al preferirlo sobre Random Forest por 9.3 pp de recall con exactitud equivalente. Reservas: la elección usó la misma validación cruzada (sesgo no medido), dos subgrupos superan el umbral y falta una matriz de costos para fijar el umbral de decisión.

Referencias

Cawley, G. C., y Talbot, N. L. C. (2010). On over-fitting in model selection and subsequent selection bias in performance evaluation. *Journal of Machine Learning Research*, *11*, 2079–2107.

Hofmann, H. (1994). *Statlog (German Credit Data)* [Conjunto de datos]. UCI Machine Learning Repository. https://doi.org/10.24432/C5NC77

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., y Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, *12*, 2825–2830.